

PoC ACL — conector CRS → Opera Cloud (bajada de reservas)

ec-demo1 · plan de la PoC · generado el 2026-09-22 a partir de docs/poc-acl/plan.md

Objetivo

Construir el camino de **bajada** de una reserva —creada, modificada o cancelada— desde un CRS simulado hasta el tenant de pruebas de Opera Cloud, siguiendo el HLA *CRS-PMS Integration - Solution* (wiki, Support Domain / Integration Subdomain).

AC: tener el **coste real** de desarrollar un conector contra la API de un tercero. Por eso el esfuerzo se registra desde el primer día en `cost-log.md`, y el del conector OHIP (`pms-integration-service`) se mide por separado del resto.

Alcance

Dentro

HLA	Proceso / capacidad
#1 (F001, F002, F014)	Proyectar Reserva: alta y modificación como el mismo <i>upsert</i> de estado, con perfil de huésped, desglose diario, referencias externas y cobros
#2 (F003)	Proyectar Cancelación , sin apunte de penalización en el folio
#3 (F004)	Proyectar Interlocutor: perfiles Travel Agent / Company en Opera desde el maestro de interlocutores
#9 (F009)	Mapeado de códigos: diccionario cadena + propiedad, versionado, con aprobación humana y propuesta por un agente
F012 (parcial)	Suspensión por causa y reanudación en bloque al resolverla
Transversal	MCP en cada servicio con operativa, notificaciones a las personas

Fuera: todo lo que sube del PMS al CRS (OOO, no-show, salida anticipada, cupo, streaming), conciliación, backfill (#10), recap, penalización en el folio, ciclo de vida completo de la integración (#8, #13), read model de causas (`integration-query-service`; se usa la vista del motor), auditoría (`audit-service`).

Decisiones tomadas

- **Sin CDC.** Controlamos `booking`, así que el aviso de cambio es un **evento de dominio** publicado por **outbox** en la misma transacción. Como en el HLA, el evento es ligero —`bookingId`, `hotel`, `version`— y **el dato se relee**.
- **Property APIs de OHIP** (`rsv`, `crm`, catálogo de configuración), no `resnotif`. Desvía del HLA: `resnotif` no revalida precio ni disponibilidad y `rsv` puede que sí. Es una incógnita a cerrar en la PoC (junto a **R19**) y se anota como tal en las conclusiones.
- **Huéspedes dentro de la reserva.** El perfil de huésped en Opera se crea con los datos que trae la reserva (F001, R11); no hay CRM de clientes en esta PoC.
- **Interlocutores en un maestro aparte (partners)**, que simula el maestro de interlocutores del lado ERP. La reserva de `booking` solo lleva el **código** del interlocutor. El proceso #3 lee

directamente del maestro, que es la alternativa de **R38**.

- **Motor:** EventConductor, el que ya corre en el clúster. La suspensión es un paso `WAIT_FOR_MESSAGE` y la reanuda un `MessageReceived` (detalle más abajo).
- **Orden de aplicación:** guarda de secuencia *compare-and-set* contra un **UDF** de la reserva en Opera (R18, R28). La versión la da `booking`.
- **El LLM nunca está en el camino del dato.** Solo propone mapeados; nada entra en vigor sin aprobación humana.
- **No se escribe nada en el tenant de Opera** (decisión del 2026-09-22). El conector se construye contra `opera-mock`, un doble de OHIP que implementa las mismas rutas y formatos de las Property APIs que usamos: token OAuth, catálogo, perfiles, reservas con UDF y cancelación. Cambiar al tenant real es cambiar la URL y las credenciales. Validar contra el tenant (R18, R19, revalidación de `rsv`) queda como paso posterior a la PoC.
- **La suspensión por causa, sin ciclos en el grafo.** El motor no admite ciclos, así que «volver a Preparar» se hace así:
 1. `Preparar` devuelve las carencias y `mapping-service` registra cada una como causa, con los procesos que esperan detrás.
 2. El proceso espera en un único `WAIT_FOR_MESSAGE` correlacionado por su propia clave.
 3. Cuando un proceso se queda sin causas abiertas, `mapping-service` le envía el mensaje.
 4. El proceso reanudado **relanza una instancia nueva**, que vuelve a leer la reserva.

Encaja con «una instancia por evento» del HLA: una instancia de más es inofensiva. Un rechazo determinista de Opera usa el mismo camino, como una causa más.

Piezas



Servicio	Nuevo / cambia	Responsabilidad	UI	MCP
booking	Cambia	CRS simulado: modelo de reserva real, versión, eventos de dominio, outbox, lectura completa, anotar referencia PMS	Sí	Sí (ampliado)
partners	Nuevo	Maestro de interlocutores: agencias, turoperadores y companies con datos fiscales, facturación y modalidad Front / No Front	Sí	Sí
integration-model	Nuevo (librería)	Modelo de negocio canónico y contratos de eventos. Ningún servicio fuera de los extremos conoce <code>booking</code> ni Opera	—	—
crs-integration-service	Nuevo	ACL del lado CRS: inbox con deduplicación, relectura, traducción al modelo canónico, router evento → proceso, worker «anotar en el CRS»	No	No
mapping-service	Nuevo	Diccionario de equivalencias y de identificadores (perfiles de interlocutor), worker <code>Preparar</code> , causas, aprobación, señal de reanudación, petición de propuesta al agente	Sí	Sí

Servicio	Nuevo / cambia	Responsabilidad	UI	MCP
pms-integration-service	Nuevo	El conector. ACL contra OHIP Property APIs: auth, catálogo, perfiles, reserva, cobros, cancelación, clasificación de errores	No	No
opera-mock	Nuevo (doble)	Simula OHIP Property APIs con estado en memoria y una UI de solo lectura para ver qué ha «llegado a Opera»	Sí	No
communication-service	Nuevo	Envío de notificaciones: plantillas, destinatarios, canal email por el relay postfix, histórico	Sí	Sí
ec-definitions	Cambia	Definiciones proyectar-reserva, proyectar-cancelacion, proyectar-interlocutor	—	—
ia-control-plane	Configuración	Alta de los MCP nuevos y del agente de mapeado	—	—

Los adaptadores (crs- y pms-integration-service) no tienen UI ni MCP, como en el HLA: traducen, y no tienen operativa propia que enseñar.

Detalle por pieza

booking como Rumbo

- **Modelo**

- Cabecera: hotel, canal, código de interlocutor, bono externo, fechas, moneda, estado y **versión** monótona.
- Habitaciones: tipo, tarifa y régimen propios del CRS, ocupación y precio por noche.
- Huéspedes por habitación, con el titular marcado.
- Cobros: tipo, forma de pago, importe, fecha e id estable.
- Los códigos son de estilo CRS, deliberadamente distintos de los de Opera, para que el ACL tenga trabajo real.

- **Eventos:** BookingCreated, BookingModified y BookingCancelled por medio de AggregateRoot.send() / popEvents(). Una tabla outbox se escribe en la misma transacción y un relay la publica a Kafka.
- **API:** lectura completa de la reserva (hace de «relectura por JDBC») y comando para anotar el reservationId de Opera.
- Adaptar la UI de Mateu (listas anidadas), el MCP y api-mcp/.../bookings-openapi.yaml.
- Alinear shared con la versión del motor desplegado (hoy 2.14.1 frente a 2.16.5).

partners

- CRUD de interlocutores con su tipo (Travel Agent, Source, Company), datos fiscales, dirección de facturación, modalidad de cobro y **versión**.
- Evento PartnerChanged por outbox, que dispara el proceso #3.
- MCP: listar, consultar, crear y modificar interlocutores, y relanzar su sincronización.

crs-integration-service

- **Inbox:** deduplica por eventId y agrupa avisos seguidos de la misma reserva.
- **Relectura** de booking y partners, y **traducción al modelo canónico**.
- **Router** (tabla evento → definición de proceso, en configuración): arranca el proceso con ProcessCreationRequested, uno por evento, con clave de correlación hotel + localizador.

- **Worker** «anotar la referencia del PMS en el CRS».

mapping-service

- **Diccionario** de cadena con excepciones por propiedad. Cubre: hotel, tipo de habitación, tarifa, régimen, estado, motivo de cancelación, forma de pago, canal / origen y tipo de interlocutor. Admite correspondencias que no son 1:1.
- **Correspondencia de identificadores**: interlocutor del CRS ↔ perfil de Opera, con la versión proyectada.
- **Worker Preparar** : resuelve **todas** las traducciones de una vez y devuelve el payload listo o la **lista completa de carencias**.
- **Causas**: cada carencia es una causa con clave `hotel/tipo/código`, con cuántos procesos esperan detrás y desde cuándo. Al aprobar una equivalencia se emite `MessageReceived`, que reanuda todos los procesos que esperaban esa causa.
- **Catálogos**: los del CRS salen de `booking`; los de Opera, de `pms-integration-service`, y se cachean.
- **Propuesta por agente**
 - Cuando hay mapeado pendiente, la UI ofrece «**Pedir propuesta al agente**». También se puede pedir desde el chat de la consola.
 - El agente usa el MCP de `mapping-service`: lee las carencias y ambos catálogos y **registra una propuesta** con una confianza por línea.
 - La propuesta queda en estado *propuesta* y el administrador la revisa, corrige, aprueba o rechaza en la UI.
 - Aprobar por MCP solo con la identidad del usuario y confirmación explícita. Nunca aprueba el agente por su cuenta.
- **Versionado**: cada aprobación crea una versión nueva, con autor y fecha.
- **MCP**: carencias y causas abiertas, catálogos, diccionario vigente, registrar propuesta y aprobar o rechazar (con la identidad del usuario).

pms-integration-service (el conector)

- **Autenticación** OHIP: token OAuth con caché y renovación, `x-app-key`, enterprise y hotel. Las credenciales van en un Secret y no en el código.
- **Catálogo** de la propiedad para el mapeado: tipos de habitación, tarifas, market / source, formas de pago, motivos de cancelación.
- **Perfiles** (`crm`): buscar por referencia externa antes de crear.
 - Huésped (provisional con los datos del titular).
 - Travel Agent / Company para el proceso #3.
- **Reserva** (`rsv`):
 - Buscar por el localizador del CRS inyectado como referencia externa.
 - Leer el UDF de secuencia y comparar con la versión que llega.
 - Crear o actualizar con el desglose diario, tarifa fija, perfiles enlazados, enrutamiento Front / No Front y los cobros como depósito referenciado a su id de origen.
 - Una versión rezagada **no se graba**.
- **Cancelación**: con el motivo mapeado, de forma idempotente (si ya estaba cancelada, no es error).
- **Clasificación de errores**:

Respuesta de OHIP	Qué hace el proceso
Timeout, 5xx, límite de caudal	Transitorio: reintento
4xx determinista	Causa y suspensión
Conflicto (la reserva ya existe)	Se lee y se actualiza

- **Pruebas** contra `opera-mock`, que reproduce las rutas y formatos de las especificaciones públicas de OHIP (`oracle/hospitality-api-docs`). No hay respuestas reales del tenant: no se le escribe nada.

communication-service

- Consume `NotificationRequested` de Kafka. Resuelve plantilla y destinatarios por tipo de aviso y hotel, envía por email a través del relay `postfix` y guarda el histórico.
- **Avisos de la PoC:**
 - Causa nueva: falta un mapeado o un interlocutor, con enlace a la UI de mapeado.
 - Propuesta del agente lista para revisar.
 - Proceso que sigue reintentando pasado el umbral (R14).
 - Rechazo determinista de Opera.
- UI de histórico y de destinatarios. MCP: consultar avisos y reenviar.
- La integración decide **qué** se notifica y **a quién**; el canal es cosa de este servicio (HLA, «Notificación a las personas»).

Definiciones de proceso (`ec-definitions`)

- **proyectar-reserva**
 1. Preparar (`mapping-service`).
 2. Si hay carencias: aviso y un `WAIT_FOR_MESSAGE` correlacionado por la clave del proceso. Al reanudarse, **relanza una instancia nueva** que vuelve a leer la reserva (el grafo no admite ciclos).
 3. Si no las hay: asegurar el perfil del huésped → grabar la reserva → anotar en el CRS.
- **proyectar-cancelacion** : esperar a que la reserva esté proyectada → preparar el motivo → cancelar en Opera → anotar en el CRS.
- **proyectar-interlocutor** : preparar → asegurar los perfiles → anotar la correspondencia → señal de reanudación para las reservas que esperaban ese interlocutor.
- **Política de reintento en los pasos externos**: backoff acotado sin límite de intentos, con aviso pasado el umbral. Ningún proceso tiene estado final de fallo.

Hitos

Una rama y un PR por hito.

Hito	Contenido	Hecho cuando
H1 ✓	<code>booking</code> ampliado, con versión, eventos y outbox; <code>shared</code> alineado (PR #21)	Crear, modificar y cancelar publican su evento con la versión correcta
H2 ✓	<code>integration-model</code> , <code>partners</code> , <code>crs-integration-service</code> con inbox, relectura y router	Un cambio en <code>booking</code> arranca un proceso con la reserva canónica
H3 ✓	<code>mapping-service</code> con el diccionario, <code>Preparar</code> , causas y reanudación; UI y MCP	Una reserva con un código sin mapear se suspende y se reanuda al aprobarlo

Hito	Contenido	Hecho cuando
H4 ✓	opera-mock, pms-integration-service contra él; definiciones #1 y #2	Punta a punta en local, con la guarda de secuencia
H5	Tenant real de Opera: aplazado , no se escriben datos en Opera	Queda como paso posterior; se documenta qué faltaría validar
H6 ✓	Proceso #3 contra el doble	Una reserva que referencia un interlocutor nuevo espera y se proyecta tras sincronizarlo
H7 ✓	communication-service y avisos	Cada tipo de aviso llega por email
H8 ✓	Propuesta de mapeado por agente	Desde la UI o el chat, el agente registra una propuesta que se aprueba y reanuda procesos
H9 ●	Despliegue en el clúster, e2e y conclusiones (manifiestos, e2e local y conclusiones hechos; falta desplegar)	Demo en ec1.mateu.io; conclusiones y coste cerrados

Pendiente de recibir

- Tenant de Opera.** Recibidos el gateway (UAT, mtce13ua, eu-frankfurt-1), x-app-key, client id / secret y enterprise **RIUE**. El 2026-09-22 se comprobó que el token OAuth (client_credentials) funciona; es de cadena **RIUC** (scope C:RIUC). **Falta el código de hotel:** las Property APIs exigen x-hotelid y con RIUC / RIUE responden 403. También falta saber qué APIs tiene activadas la app Riu_Hotel_Cliente_OHIP. Solo hace falta para la validación posterior contra el tenant real: la PoC se completa contra opera-mock. Las credenciales no van en el repo: van a deploy/.secrets/ y de ahí a un Secret. **Rotar el client secret al cerrar la PoC.**
- Configuración del hotel de pruebas:**
 - Qué catálogos tiene y si se pueden leer por API.
 - Si hay **UDF libres** en la reserva para la guarda de secuencia.
 - Si hay perfiles de interlocutor ya creados.

Entregables

- El código de los hitos, en PRs.
- cost-log.md, con el coste del conector OHIP separado del resto.
- Conclusiones:** las incógnitas del HLA que la PoC cierra (R28, R38, la suspensión por causa sobre el motor), las que solo puede cerrar el tenant real (R18, R19, revalidación con rsv) y los problemas nuevos que haya sacado a la luz, como entrada para el DT.